SMART CONTRACT AUDIT

Conducted for: C2C

VulSight
25/7/2023

Contents

Executive Summary:

The client contacted our Firm to conduct a smart contract audit on their Solidity based smart contract. The smart contract was deployed on Mumbai MATIC Test Net. The smart contract served as a mechanism to transfer funds based on a voucher redeeming functionality within the web application. Most of the functions in this contract are highly centralized in nature and can only be called by delegate or an owner of the contract. The "C2CContract" is a multi-functional contract, providing a robust platform for managing various tokens within a controlled, secure environment. It had “Owner” and “Delegate” roles within it. The audit was performed using both manual code analysis and automated security tools. The audit is performed on a best effort basis. Few vulnerabilities of low and medium severity were detected in the audit. No significant Vulnerabilities were detected within the smart contract code during the course of this audit. All of the findings of the audit is detailed inside this report.

Project Background:

The smart contract is written in solidity. This token includes roles such as “Owner” and “Delegate”. The contract was deployed on the Mumbai Testnet.

Audit Scope:

Deployed Address	**0x208eeBEb0cc54A04e439C868ff8906E752c9E7f5**
Chain	**Mumbai Testnet**
Language	**Solidity**
Audit Completion Date	**25/7/23**

Audit Summary:

- **Owner Assignment:** The contract designates an owner upon creation. The owner has the authority to carry out privileged operations.
- **Delegate Management:** The owner can appoint other addresses as "delegates", who are also granted permission to perform specific functions.
- **Token Whitelisting:** The contract owner can add tokens to a whitelist, enabling them for transactions within this contract.
- **Token Deposit and Withdrawal:** Addresses with sufficient permissions (the owner and any delegates) can deposit or withdraw whitelisted tokens from the contract. Both operations generate an event logging the transaction details.
- **Token Transfers:** The contract owner or delegates can directly send assets to a specific address.

- **On-Chain Stats:** The contract keeps track of the total amounts of each token type deposited and withdrawn, allowing for historical data access.
- **MATIC Management:** The contract can also receive, store, and withdraw the native blockchain cryptocurrency, MATIC, allowing for greater versatility and functionality.

We found:

- **0 Critical risk vulnerabilities**
- **0 High risk vulnerabilities**
- **2 Medium risk vulnerabilities**
- **3 Low risk vulnerabilities**

Severity Definitions:

Risk Level	Description
Critical	**Any kind of vulnerability that could lead to direct token or monetary loss**
High	**Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.**
Medium	**Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss**
Low	**Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract**

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Logical Vulnerabilities	Passed
Function visibility not explicitly declared	Not Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Not Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract which was written in Solidity programming language. The code is well indented and clear in nature. While the Code lacked commenting.

Audit Findings:

Critical vulnerabilities:

0 critical vulnerabilities were found in the smart contract.

High vulnerabilities:

0 high vulnerabilities were found in the smart contract.

Medium vulnerabilities:

2 medium vulnerabilities were found in the smart contract.

Receive function logic error:

The smart contract has a receive function to deal with the transfers of MATIC directly to the smart contract. But the logic of the function seems to be flawed. As rather than incrementing the Deposit variable it is incrementing the Withdrawal variable. This could cause a calculation error in the contract as when MATIC is actually sent to the contract, the Deposit variable should be incremented.

```
receive() external payable {
     balances["Matic"] += msg.value;
     totalWithdrawn["Matic"]+=msg.value;
 }
```

No function to transfer Ownership:

The smart contract, as currently designed, has an inherent vulnerability due to its lack of functionality for transferring ownership. The contract assigns ownership upon creation, assigning this status to the address that deploys the contract. This ownership role is permanent and irrevocable, with no provision included for transferring it to another address.

This lack of ownership transfer capability presents potential operational risks and challenges:

- **Single Point of Failure:** Should the private key for the owner's address become lost or compromised, it will be impossible to perform administrative actions requiring owner permissions, rendering the contract effectively frozen and unusable.

- **Inability to Transfer Control:** If the original owner wishes to step down, retire, or otherwise hand over control of the contract to another party for any reason, they are unable to do so. This

limits the lifecycle and adaptability of the contract and can pose significant business continuity risks.

Low vulnerabilities:

3 low vulnerabilities were found in the smart contract.

Function Visibility not specified:

No Function Visibility is specified for **addDelegate** and **removeDelegate**. In Solidity, function visibility is defined by specifying public, external, internal, or private. The addDelegate and removeDelegate functions are public by default since no visibility is specified. Although these functions are protected by the onlyOwner modifier, it's good practice to explicitly state function visibility.

Absence of Emergency Stop Mechanism:

The contract does not include an 'emergency stop' or 'circuit breaker' mechanism. This would allow the contract's owner to pause certain functionalities in case a bug or a security breach is detected.

Centralization of Power:

Much of the power is centralized to the contract's owner and the designated delegates, including functions like whitelisting tokens and depositing/withdrawing tokens. The centralization is a point of vulnerability and could pose risks if the owner's account is compromised.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.

Conclusion:

The smart contract was tested extensively both automatically and manually. Few Vulnerabilities were discovered within it. It is recommended these vulnerabilities be resolved. It is also recommended that additional security measures may be undertaken to ensure future security such that of a creation of a bug bounty program.

It is also recommended to use a proxy mechanism in the smart contract to ensure that the smart contract code can be modified in the future.